Venus SwapRouter Audit Report

Prepared by Cyfrin

Version 2.0

Lead Auditors

SBSecurity (Blckhv, Slavcheww)

Jorge

January 20, 2026

Contents

1 About Cyfrin **2**

2 Disclaimer **2**

3 Risk Classification **2**

4 Protocol Summary **2**
4.1 SwapRouter Contract . . . 2

5 Audit Scope **3**

6 Executive Summary **3**

7 Findings **4**
7.1 Low Risk . . . 4
7.1.1 Loss of user funds due to unreturned leftover tokens in amount . . . 4
7.1.2 Supply Operations Lack vToken Exchange Rate Protection . . . 5
7.1.3 Missing recipient argument for supply operations . . . 6
7.1.4 VBNB supply operations permanently blocked when transfer is paused a different from the normal VBep20 markets . . . 7
7.2 Informational . . . 9
7.2.1 `_handleTokenInput` should support only non-native `tokenIn` . . . 9

1 About Cyfrin

Cyfrin is a Web3 security company dedicated to bringing industry-leading protection and education to our partners and their projects. Our goal is to create a safe, reliable, and transparent environment for everyone in Web3 and DeFi. Learn more about us at cyfrin.io.

2 Disclaimer

The Cyfrin team makes every effort to find as many vulnerabilities in the code as possible in the given time but holds no responsibility for the findings in this document. A security audit by the team does not endorse the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the solidity implementation of the contracts.

3 Risk Classification

	Impact: High	**Impact: Medium**	**Impact: Low**
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

4 Protocol Summary

Venus Protocol is a decentralized algorithmic money market protocol which enables users to supply crypto assets as collateral to earn interest, borrow against their collateral through an over-collateralized loan system, and mint VAI—a synthetic stablecoin pegged to $1 USD.

When users deposit assets, they receive vTokens representing their share of the liquidity pool, with interest rates determined algorithmically based on supply and demand utilization curves. The protocol is governed by XVS token holders who can propose and vote on parameter changes including collateral factors, interest rate models, and supported assets.

Venus employs standard DeFi risk management mechanisms including liquidations when collateral ratios fall below required thresholds, and the protocol architecture follows a Compound-style design with a Comptroller (implemented as a Diamond proxy) that enforces risk parameters across all vToken markets.

4.1 SwapRouter Contract

The `SwapRouter` is an upgradeable periphery contract for Venus Protocol that enables users to perform atomic swap-and-supply and swap-and-repay operations in a single transaction. Users can supply collateral to Venus markets or repay outstanding debts using any input token (including native BNB), with the contract handling the intermediate swap via a configurable SwapHelper contract.

The contract supports six primary operations: `swapAndSupply`, `swapNativeAndSupply`, `swapAndRepay`, `swapNativeAndRepay`, `swapAndRepayFull`, and `swapNativeAndRepayFull` with the "full" variants designed to repay a user's entire debt position. Key safety features include slippage protection via `minAmountOut` parameters, market validation against the Comptroller, reentrancy guards, and fee-on-transfer token support through balance-before/after accounting.

The contract handles the native vBNB market specially since it doesn't support `mintBehalf`, instead minting vBNB to itself and transferring the resulting vTokens to the user. Owner-only sweep functions allow recovery of any leftover tokens that may accumulate from rounding or failed operations.

5 Audit Scope

The audit scope was limited to:

```
contracts/SwapRouter/SwapRouter.sol
```

6 Executive Summary

Over the course of 3 days, the Cyfrin team conducted an audit on the Venus SwapRouter smart contracts provided by Venus. In this period, a total of 5 issues were found.

The findings consist of 4 Low severity issues and one informational.

Summary

Project Name	Venus SwapRouter
Repository	venus-periphery
Commit	aeb20e136743...
Fix Commit	0f8abbe710b1...
Audit Timeline	Jan 12th - Jan 14th, 2026
Methods	Manual Review

Issues Found

Critical Risk	0
High Risk	0
Medium Risk	0
Low Risk	4
Informational	1
Gas Optimizations	0
Total Issues	5

Summary of Findings

[L-1] Loss of user funds due to unreturned leftover tokens in amount	Acknowledged
[L-2] Supply Operations Lack vToken Exchange Rate Protection	Acknowledged
[L-3] Missing recipient argument for supply operations	Acknowledged
[L-4] VBNB supply operations permanently blocked when transfer is paused a different from the normal VBep20 markets	Acknowledged
[I-1] `_handleTokenInput` should support only non-native `tokenIn`	Resolved

7 Findings

7.1 Low Risk

7.1.1 Loss of user funds due to unreturned leftover tokens in amount

Description: The `SwapRouter::_performSwap` function fails to return unused input tokens to users after executing swaps through the `SwapHelper` contract. When a DEX swap uses less than the full input amount (due to partial fills, better routing, or slippage protection), the leftover tokens are either:

1. Stuck in the `SwapHelper` contract (if the backend doesn't include a sweep call for the input token)
2. Stuck in the `SwapRouter` contract (if the backend includes the sweep but `SwapRouter` doesn't refund)

In both scenarios, users permanently lose their unused tokens, which can only be recovered by the contract owner via the `sweepToken` function.

The `_performSwap` function only tracks the `tokenOut` balance change but ignores any `tokenIn` that may be returned:

```
function _performSwap(
    address tokenIn,
    address tokenOut,
    uint256 amountIn,
    uint256 minAmountOut,
    bytes calldata swapCallData
) internal returns (uint256 amountOut) {
    // If tokens are the same, no swap needed
    if (tokenIn == tokenOut) {
        if (minAmountOut > 0 && amountIn < minAmountOut) {
            revert InsufficientAmountOut(amountIn, minAmountOut);
        }
        return amountIn;
    }

    // Transfer tokens to SwapHelper
    IERC20Upgradeable(tokenIn).safeTransfer(address(SWAP_HELPER), amountIn); //<------

    // Record balance before swap
    uint256 balanceBefore = IERC20Upgradeable(tokenOut).balanceOf(address(this));

    // Execute swap using SwapHelper multicall
    (bool success, ) = address(SWAP_HELPER).call(swapCallData);
    if (!success) revert SwapFailed();

    // Calculate amount received
    uint256 balanceAfter = IERC20Upgradeable(tokenOut).balanceOf(address(this));
    amountOut = balanceAfter - balanceBefore;

    if (amountOut == 0) revert NoTokensReceived();
    if (minAmountOut > 0 && amountOut < minAmountOut) {
        revert InsufficientAmountOut(amountOut, minAmountOut);
    }

    return amountOut;
}
```

Link

As you can see the contract is sending the `amountIn` but in case that the whole `amountIn` is not used in the swap the funds will be stuck in the `swapHelper` and will not be returned to the user

Impact: Users lose any portion of their input tokens that aren't consumed by the DEX swap

Proof of Concept: As a proof of concept in the `LeverageStartegiesManager.ts` we can see an encode call function which is returning the tokenIn left behind to the caller contract.

```
// Encode  function call
const approveMaxCall = swapHelper.interface.encodeFunctionData("approveMax", [tokenIn, dexTarget]);
const genericCall = swapHelper.interface.encodeFunctionData("genericCall", [dexTarget, dexCalldata]);
const tokenInSweepCall = swapHelper.interface.encodeFunctionData("sweep", [tokenIn,
↪  positionSwapper]);// <------
const tokenOutSweepCall = swapHelper.interface.encodeFunctionData("sweep", [tokenOut,
↪  positionSwapper]);
```

[Link]

As we can see in the encode call above to the `swapHelper` The `tokenIn` not used in the dex swap is being returned to the caller contract

Recommended Mitigation: Because `amountIn` is forwarded in full to `SwapHelper`, implement **automatic post-swap cleanup and refund** so leftover tokens can never be stranded due to missing calldata steps.

1. **Enforce deterministic cleanup after every multicall**
   - After `SwapHelper.multicall` completes, **always**:
   - `sweep(tokenIn, address(this))` from `SwapHelper` to `SwapRouter`.
2. Then, in `SwapRouter`, **refund any leftover** `tokenIn` held by the router to `msg.sender`.

```
function _performSwap(
    address tokenIn,
    address tokenOut,
    uint256 amountIn,
    uint256 minAmountOut,
    bytes calldata swapCallData
) internal returns (uint256 amountOut) {

+   uint256 refundAmount = balanceInAfter - balanceInBefore;

+   // Refund any unused input tokens to the user
+   if (refundAmount > 0) {
+      IERC20Upgradeable(tokenIn).safeTransfer(msg.sender, refundAmount);
+      emit TokenRefunded(msg.sender, tokenIn, refundAmount); // Add this event
+   }
}
```

This removes the need for backend-generated manual `sweep` calls in `swapCalldata`, makes swaps "safe by default," and matches patterns required by push/settle style integrations (e.g., UniV4-like flows) where deterministic cleanup is essential.

Venus: Acknowledged; we've already requested PCS to add a field in their API that allows disabling partial fulfillment (i.e., if partial fulfillment occurs, the transaction should revert).

Handling this on our side isn't simple because we don't know in advance what the refund token will be. For example, in a multi-hop swap such as USDC → USDT → CAKE, if partial fulfillment occurs during the second hop, the refund token would be USDT and there's no way for us to know that without decoding the call data.

7.1.2 Supply Operations Lack vToken Exchange Rate Protection

Description: The `SwapRouter` contract implements slippage protection only for the swap operation but don't implement protections users against vToken exchange rate fluctuations during the supply step. The `swapAndSupply()` and `swapNativeAndSupply()` functions perform a two-step conversion process:

1. Swap: `tokenIn` → `tokenOut` (protected by `minAmountOut` parameter)

2. Supply: `tokenOut` → `vTokens` (NO slippage protection)

The `_performSwap()` function validates that the received amount meets the `minAmountOut` threshold:

```
    if (minAmountOut > 0 && amountOut < minAmountOut) {
            revert InsufficientAmountOut(amountOut, minAmountOut);
        }
```

[Link]

However, the _supply() function performs the conversion to vTokens without any validation of the final vToken amount received:

```
function _supply(
        address vToken,
        address underlyingToken,
        uint256 amount
    ) internal returns (uint256 amountSupplied) {
        if (vToken == address(NATIVE_VTOKEN)) {
            // Handle native token supply
            IWBNB(underlyingToken).withdraw(amount);
            NATIVE_VTOKEN.mint{ value: amount }(); //<--------
            amountSupplied = amount;

            // Transfer vBNB tokens to user (only needed for NATIVE_VTOKEN since it doesn't support
            ↪  mintBehalf)
            uint256 vTokenBalance = IERC20Upgradeable(vToken).balanceOf(address(this));
            if (vTokenBalance > 0) {
                IERC20Upgradeable(vToken).safeTransfer(msg.sender, vTokenBalance);
            }
        } else {
            // Handle ERC20 token supply
            IERC20Upgradeable(underlyingToken).forceApprove(vToken, amount);
            uint256 errorCode = IVToken(vToken).mintBehalf(msg.sender, amount);//<--------
            if (errorCode != 0) revert SupplyFailed(errorCode);
            amountSupplied = amount;
        }
    }
```

[Link]

The function returns `amountSupplied` which represents the underlying tokens supplied, not the `vTokens` received. The actual `vToken` amount received depends on the current exchange rate, which can fluctuate:

```
exchangeRate = (totalCash + totalBorrows + flashLoanAmount - totalReserves) / totalSupply
```

[Link]

Impact: Users can suffer unexpected losses due to unfavorable vToken exchange rate movements between the time they calculate their expected output and when their transaction executes.

Recommended Mitigation: Implement end-to-end slippage protection by adding a `minVTokensOut` parameter to validate the final vToken amount received

Venus: Acknowledged; in our implementation, slippage protection is enforced for the swap step to ensure users receive at least the minimum expected amount of the output token.

For the supply step, we do not require additional slippage protection for vTokens because the vTokens are minted directly to the user based on the current exchange rate at the time of the transaction using the mintBehalf function.

7.1.3 Missing recipient argument for supply operations

Description: `VToken` has a recipient parameter, which allows the suppliers to give an arbitrary address for different purposes - integrators, non-compliant contracts, etc. However, the supply wrapper in the `SwapRouter` doesn't allow users to pass a custom recipient for the minted `VTokens`.

As we see, `mintBehalf` uses the `recipient` argument to grant the receipt tokens:

VBep20

```
function mintBehalf(address receiver, uint mintAmount) external returns (uint) {
    (uint err, ) = mintBehalfInternal(receiver, mintAmount);
    return err;
}
```

While using the `msg.sender` as the underlying asset supplier: VBep20

```
function mintBehalfInternal(address receiver, uint mintAmount) internal nonReentrant returns (uint,
↪  uint) {
    uint error = accrueInterest();
    if (error != uint(Error.NO_ERROR)) {
        // accrueInterest emits logs on errors, but we still want to log the fact that an attempted
        ↪  mintBehalf failed
        return (fail(Error(error), FailureInfo.MINT_ACCRUE_INTEREST_FAILED), 0);
    }

    // mintBehalfFresh emits the actual Mint event if successful and logs on errors, so we don't need to
    return mintBehalfFresh(msg.sender, receiver, mintAmount);
}

function mintBehalfFresh(address payer, address receiver, uint mintAmount) internal returns (uint,
↪  uint) {
        ...MORE CODE
    vars.actualMintAmount = doTransferIn(payer, mintAmount);
```

Impact: Integrators such as vaults, account abstraction relayers, or escrow contracts cannot atomically route receipt tokens to end users. They must perform an additional transfer, increasing gas cost, complexity, and failure surface. Some non-compliant contracts cannot forward tokens at all, making integration impossible.

Recommended Mitigation: Consider exposing a `recipient` argument to the `swapNativeAndSupply` and `swapAndSupply`.

Venus: Acknowledged; this behavior is intentional. The SwapRouter is designed to supply only on behalf of the caller (msg.sender) and does not support specifying a custom recipient for vTokens. This approach keeps the contract logic simple and secure, ensuring that only the user initiating the transaction receives the vTokens.

Additionally, since the SwapRouter requires backend-signed calldata as one of its parameters, it's effectively limited to internal Venus usage. This design makes third-party integrations impractical, external projects that wish to integrate should do so directly via the vToken contract instead.

7.1.4 VBNB supply operations permanently blocked when transfer is paused a different from the normal VBep20 markets

Description: The SwapRouter contract will be DoS for supply operations when the `Action.TRANSFER` is paused in the VBNB token. This creates a scenario where VBNB supply functionality becomes completely unavailable while VBep20 markets continue to function normally. The root cause is the flow for VBNB supply, the SwapRouter must:

1. Mint vBNB tokens to itself
2. Transfer the vBNB tokens to the user

```
function _supply(
        address vToken,
        address underlyingToken,
        uint256 amount
    ) internal returns (uint256 amountSupplied) {
        if (vToken == address(NATIVE_VTOKEN)) {
            // Handle native token supply
            IWBNB(underlyingToken).withdraw(amount);
```

```
            uint256 vTokenBalanceBefore = IERC20Upgradeable(vToken).balanceOf(address(this));
            NATIVE_VTOKEN.mint{ value: amount }();// <-------
            uint256 vTokenBalanceAfter = IERC20Upgradeable(vToken).balanceOf(address(this));

            uint256 mintedAmount = vTokenBalanceAfter - vTokenBalanceBefore;
            amountSupplied = amount;

            // Transfer only the newly minted vBNB tokens to user (only needed for NATIVE_VTOKEN since
            ↪  it doesn't support mintBehalf)
            if (mintedAmount > 0) {
                IERC20Upgradeable(vToken).safeTransfer(msg.sender, mintedAmount);//<-------
            }
        } else {
            // Handle ERC20 token supply
            IERC20Upgradeable(underlyingToken).forceApprove(vToken, amount);
            uint256 errorCode = IVToken(vToken).mintBehalf(msg.sender, amount);
            if (errorCode != 0) revert SupplyFailed(errorCode);
            amountSupplied = amount;
        }
    }
```

This two-step process introduces an additional permission check (`transferAllowed`) that is not present in the VBep20 flow, where `mintBehalf` directly credits tokens to the user.

The `safeTransfer` call triggers `VToken.transfer()`, which calls:

```
uint allowed = comptroller.transferAllowed(address(this), src, dst, tokens);
```

[Link]

This then checks:

```
checkProtocolPauseState();
checkActionPauseState(vToken, Action.TRANSFER); // <-------  REVERTS when TRANSFER is paused
```

[Link]

This check will not be present in the VBep20 supply flow since there is not transfer.

Impact: Complete DoS of `SwapRouter` for supply native token operations when the VBNB transfer token is in paused a different from the VBep20 that is not doing this check

Recommended Mitigation: There is not reliable way to fix this issue without modified the VBNB contract.

Venus: Acknowledged; the vBNB market is not upgradeable and does not support the mintBehalf functionality. Therefore, this two-step flow—minting vBNB to the contract and then transferring to the user—is required for native token supply. This limitation is inherent to the current vBNB contract design and cannot be addressed at the SwapRouter level.

7.2 Informational

7.2.1 `_handleTokenInput` should support only non-native `tokenIn`

Description: `_handleTokenInput` is used in `swapAndSupply`, `swapAndRepay`, and `swapAndRepayFull`, while neither of them works when `tokenIn = NATIVE_TOKEN_ADDR`.

However, `_handleTokenInput` has a case which expects a native BNB token to wrap it:

```
function _handleTokenInput(address tokenIn, uint256 amountIn) internal returns (uint256 actualAmountIn)
↪  {
    if (tokenIn == NATIVE_TOKEN_ADDR) {
        // Native token - should use msg.value
        if (msg.value != amountIn) revert InsufficientBalance();
        WRAPPED_NATIVE.deposit{ value: msg.value }();
        return msg.value;
    }
}
```

The problem is that `tokenIn`, once wrapped, is never updated to reference `WRAPPED_NATIVE`. Since BNB is not ERC20-compatible and lacks a `transfer` function, all downstream ERC20 transfers revert when the value is forwarded to `SwapHelper`.

```
function _performSwap(
    address tokenIn,
    address tokenOut,
    uint256 amountIn,
    uint256 minAmountOut,
    bytes calldata swapCallData
) internal returns (uint256 amountOut) {
    // If tokens are the same, no swap needed
    if (tokenIn == tokenOut) {
        if (minAmountOut > 0 && amountIn < minAmountOut) {
            revert InsufficientAmountOut(amountIn, minAmountOut);
        }
        return amountIn;
    }

    // Transfer tokens to SwapHelper
    IERC20Upgradeable(tokenIn).safeTransfer(address(SWAP_HELPER), amountIn);//@audit for tokenIn =
    ↪  native, this reverts
        ...MORE CODE
}
```

Impact: No direct impact in the current deployment since all entry points invoking _handleTokenInput are non-payable, making the native branch unreachable. However, the presence of incomplete native-token handling creates misleading assumptions and could introduce failures if future changes add payable entrypoints or reuse this helper elsewhere.

Recommended Mitigation: Remove the native token handling in `_handleTokenInput` and support only proper ERC20 assets:

```
function _handleTokenInput(address tokenIn, uint256 amountIn) internal returns (uint256 actualAmountIn)
↪  {
-   if (tokenIn == NATIVE_TOKEN_ADDR) {
-       // Native token - should use msg.value
-       if (msg.value != amountIn) revert InsufficientBalance();
-       WRAPPED_NATIVE.deposit{ value: msg.value }();
-       return msg.value;
-   } else {
        // ERC20 token - measure actual amount received
        IERC20Upgradeable token = IERC20Upgradeable(tokenIn);
```

```
        uint256 balanceBefore = token.balanceOf(address(this));
        token.safeTransferFrom(msg.sender, address(this), amountIn);
        uint256 balanceAfter = token.balanceOf(address(this));

        actualAmountIn = balanceAfter - balanceBefore;
        return actualAmountIn;
-   }
}
```

Venus: Fixed in commit 0f8abbe.

Cyfrin: Verified.